

Session 4 · Deck 4.A2

Optimization on an Interval

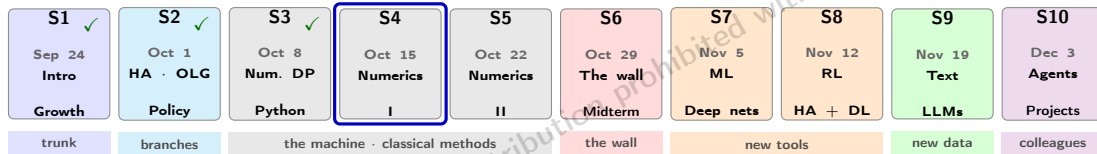
Zhigang Feng

Thursday, October 15, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 4.A2 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block A: 4.A1 root-finding · 4.A2 optimization on an interval · 4.A3 optimization in many dimensions · 4.A4 differentiation.

Twenty-Six Evaluations, or a Hundred Thousand

Maximize a concave function on $[0, 10]$ to an accuracy of 10^{-4} .

accuracy	grid	golden
10^{-4}	100,000	26
10^{-6}	10 million	36
10^{-8}	1 billion	46

Each extra decimal digit costs the grid a *factor* of ten and costs golden section **five more evaluations**.

Where this bites

3.A2 and 3.A4 did the inner maximization by searching all N grid points, every state, every sweep. That was 57 million comparisons for one deterministic RBC solve.

And the grid search does not merely cost more. It also **cannot return an answer between two grid points** — which is the error no amount of iterating removed.

What Licenses the Method: Unimodality

Golden section is a *bracketing* method, like bisection — and it needs the same kind of guarantee. Its version is **unimodality**: one interior maximum, no flat spots, no second hill.

In the Bellman problem the objective is

$$u(y(k, z) - k') + \beta \hat{v}(k'),$$

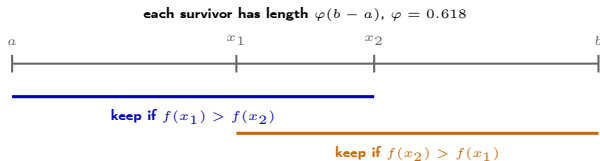
so it is concave as long as u is concave *and the interpolant $\hat{v}$ is*. Concave $\Rightarrow$ unimodal $\Rightarrow$ the bracket can be shrunk safely.

And it does

Linear interpolation **preserves concavity**: the chord of a concave function lies below it, so the interpolant is concave too. Measured, the worst second difference of a linearly interpolated concave function is 4×10^{-16} — zero.

So the humblest interpolation scheme is exactly the one that guarantees the property this method needs. Cubic splines, which are more accurate, do **not** — 4.B1 measures that.

Golden Section: Shrink the Bracket, Reuse a Point



Place the two interior points symmetrically at $a + \tau(b - a)$ and $b - \tau(b - a)$. Discard the outer piece on the losing side. **One of the two survivors is already an interior point of the new bracket**, so each later step costs only *one* new evaluation.

Why that ratio

Recycling requires the new interior point to sit where the old one already is:

$$\tau = (1 - \tau)^2 \implies \tau^2 - 3\tau + 1 = 0,$$
$$\tau = \frac{3 - \sqrt{5}}{2} = 0.382, \quad \varphi = 1 - \tau = 0.618.$$

The golden ratio is not mysticism here. It is the unique ratio that makes the picture self-similar, so that one function value can be reused forever.

Adapted from QM *Numerical Methods*, "Golden Section Search", "The Efficiency Hack" and "Why the Golden Ratio?".

The Cost, and a Correction

After n evaluations the bracket has length $\varphi^n(b-a)$, so reaching a tolerance ε takes

$$n \approx \frac{\ln(\varepsilon/(b-a))}{\ln \varphi} = O\left(\log \frac{1}{\varepsilon}\right).$$

Measured on $[0, 10]$: 26, 36 and 46 evaluations at $\varepsilon = 10^{-4}, 10^{-6}, 10^{-8}$, against a theoretical 23.9, 33.5 and 43.1.

It is not $O(\log M)$

The cost has **nothing to do with the grid size M** . A source table reporting “ $O(\log M)$, about 24 evaluations at $M = 1000$ ” is quoting a *tolerance* count and labelling it a grid count.

The distinction matters: refine your state grid tenfold and the grid search gets ten times more expensive, while golden section costs **exactly the same**.

Brent, and When You Have Derivatives

Brent's minimizer is the same idea as brentq: fit a parabola through three points and jump to its vertex, but fall back to a golden-section step whenever the parabola is unhelpful. Superlinear on smooth problems, never worse than golden section.

Newton, if f' and f'' are available, solves $f'(x) = 0$:

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)},$$

quadratically convergent — and, exactly as in 4.A1, only locally safe.

Maximizing is not root-finding

Applying a root-finder to f' finds *stationary points*: maxima, minima and inflections alike. The sign of f'' is what tells them apart, and a bracketing maximizer never needs to ask.

In a Bellman loop the objective changes every sweep, so its derivatives change too. That is why golden section, which needs neither, is the workhorse.

Pay-Off: The Same Model, With Continuous Choice

Take the quarterly RBC of this session, and change exactly one thing: replace the grid search over k' with golden section on the interval, evaluating $\hat{v}$ by linear interpolation.

	grid	golden
policy updates	66	66
wall time	0.25 s	1.11 s
worst $\log_{10} \mathcal{E}$	-1.55	-2.77
mean $\log_{10} \mathcal{E}$	-2.18	-3.47

1.2 decades of accuracy, for four and a half times the wall clock — and not one extra sweep.

Where the gain comes from

Not from iterating better. The iteration count is identical. It comes from the policy no longer being forced onto a grid point.

The wall-clock cost is an artifact of doing golden section in a Python loop. The *evaluation* count — 18.9 million objective evaluations against 2.4 billion comparisons — goes the other way, by a factor of 125.

The Same Defect, Invisible

In 3.A4's model ($\delta = 1$) the grid search produced a visible **staircase** — 36 consecutive states sharing 11 distinct policy values.

Run the identical diagnostic here, at $\delta = 0.025$, and it looks fine: 36 states, **35** distinct values. The policy is close to the 45-degree line, so consecutive states land on consecutive grid points and the steps are invisible.

But the residual did not improve. It is still $10^{-1.55}$, and continuous choice still buys 1.2 decades.

The lesson

The error was never the picture. It was always the $\pm h/2$ quantization, and whether you can see it depends on the slope of the policy — that is, on the calibration.

Diagnose with the residual, not the plot. A smooth-looking policy function is not evidence of anything; 3.A1's Santos argument is why the residual is evidence.

Takeaways

- Golden section reaches 10^{-8} on an interval in **46 evaluations**; a grid fine enough to match it needs a billion. Each extra digit costs five more evaluations, not a factor of ten.
- Its cost is $O(\log(1/\epsilon))$ and **independent of the state grid**. Refining the state grid does not make the inner maximization harder.
- The golden ratio is the unique one that makes the bracket self-similar, so one function value can be recycled at every step.
- What licenses the method is **unimodality**, and in the Bellman problem that comes from concavity of u *and* of the interpolant — which linear interpolation supplies and cubic splines do not.
- Measured pay-off: **1.2 decades** of Euler accuracy, with the iteration count unchanged. And the defect it removed was invisible in the policy plot at this calibration — **report the residual**.

Bridge: When the Choice Is Not a Scalar

Everything here relied on the choice being **one number on an interval**. Bracketing works because an interval has two ends.

Add labour supply and the household chooses (k', n) . Add a second asset and it chooses (a', b') . Estimate the model and you are choosing a parameter vector. In $\mathbb{R}^N$ there is no bracket to shrink — and the methods that replace it need derivatives, or clever ways of avoiding them.

Next (4.A3)

Optimization in many dimensions: gradients and Hessians, why a Newton step costs $O(N^3)$, BFGS worked through by hand, line search, Nelder–Mead, and what changes when constraints bind.